Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive plots in Python.
This cheat sheet provides a quick reference from basic to advanced usage, covering essential features for data science, machine learning, and scientific computing.

Star on GitHub 10.7k | Follow us on X | Buy Me a Coffee

Getting Started

Importing

```
import matplotlib.pyplot as plt  # Core plotting library
import numpy as np               # For numerical operations
```

Basic Plot

```
x = np.linspace(0, 10, 100)      # 100 points between 0 and 10
y = np.sin(x)                    # Sine function values
```

```
plt.plot(x, y)                # Create a line plot
```

Plot Types

Line Plot

```
plt.plot(x, y)                 # Line plot of y vs x
plt.title("Sine Wave")        # Set title
plt.xlabel("x-axis")          # Label x-axis
plt.ylabel("y-axis")          # Label y-axis
plt.grid(True)                # Show gridlines
plt.show()
```

Scatter Plot

```
plt.scatter(x, y)             # Scatter plot
plt.title("Scatter Plot")
plt.show()
```

Bar Plot

```
categories = ['A', 'B', 'C']
values = [10, 20, 15]
plt.bar(categories, values)   # Vertical bar chart
plt.title("Bar Chart")
plt.show()
```

Horizontal Bar Plot

```
plt.barh(categories, values)  # Horizontal bar chart
plt.title("Horizontal Bar Chart")
plt.show()
```

Histogram

```python
data = np.random.randn(1000)    # Random normal distribution
plt.hist(data, bins=30)         # Histogram with 30 bins
plt.title("Histogram")
```

Pie Chart

```python
sizes = [25, 35, 20, 20]
labels = ['A', 'B', 'C', 'D']
plt.pie(sizes, labels=labels, autopct='%1.1f%%')  # Pie chart with % labels
plt.title("Pie Chart")
plt.show()
```

Customization

Title	`plt.title("Title")`	Set the title of the plot
X/Y Labels	`plt.xlabel("X")`, `plt.ylabel("Y")`	Label the axes
Grid	`plt.grid(True)`	Show grid
Legend	`plt.legend(["line1"])`	Add legend
Line Style	`plt.plot(x, y, linestyle='--')`	Dashed line
Color	`plt.plot(x, y, color='green')`	Set line color
Marker	`plt.plot(x, y, marker='o')`	Show markers on points
Axis Limits	`plt.xlim(0, 10)`, `plt.ylim(-1, 1)`	Set axis range

Tick Labels	`plt.xticks([...])`, `plt.yticks([...])`	Customize tick positions
Text Annotation	`plt.text(5, 0, "Midpoint")`	Add text at a specific coordinate
Arrow	`plt.annotate("Peak", xy=(7, 1), xytext=(6, 1.5), arrowprops=dict(arrowstyle="->"))`	Add annotation arrow
Style Sheets	`plt.style.use('ggplot')`	Use predefined styles like `seaborn`, `bmh`

Subplots & Layouts

Multiple Subplots

```
fig, axs = plt.subplots(2, 2)        # Create 2x2 grid of subplots
axs[0, 0].plot(x, y)                 # Top-left subplot
axs[0, 1].scatter(x, y)              # Top-right subplot
axs[1, 0].bar(categories, values)    # Bottom-left subplot
axs[1, 1].hist(data)                 # Bottom-right subplot
plt.tight_layout()                   # Adjust spacing to prevent overlap
plt.show()
```

Figure Size

```
plt.figure(figsize=(10, 5))         # Set figure size (width, height in inches)
```

Advanced Visualizations

Heatmap

```
data = np.random.rand(10, 10)       # Random 10x10 matrix
plt.imshow(data, cmap='hot', interpolation='nearest')  # Display as image
plt.colorbar()                      # Show color scale
plt.title("Heatmap")
plt.show()
```

Contour Plot

```
X, Y = np.meshgrid(x, x)
Z = np.sin(X) * np.cos(Y)
plt.contour(X, Y, Z)                # Contour lines
plt.title("Contour Plot")
plt.show()
```

3D Plot

```
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')  # 3D subplot
ax.plot3D(x, y, np.cos(x))                  # 3D line
plt.title("3D Plot")
plt.show()
```

Working with Images

```
import matplotlib.image as mpimg
img = mpimg.imread('image.jpg')     # Load image
plt.imshow(img)                     # Display image
plt.axis('off')                     # Hide axes
```

```python
plt.title("Image Display")
plt.show()
```

For Machine Learning

Plotting Loss vs Epoch

```python
epochs = range(1, 11)
loss = [0.9, 0.7, 0.5, 0.4, 0.3, 0.25, 0.2, 0.18, 0.15, 0.13]
plt.plot(epochs, loss)
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

Confusion Matrix (Heatmap)

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns

y_true = [0, 1, 2, 2, 0]
y_pred = [0, 0, 2, 2, 1]
cm = confusion_matrix(y_true, y_pred)    # Create confusion matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')  # Heatmap visualization
plt.title("Confusion Matrix")
plt.show()
```

ROC Curve

```python
from sklearn.metrics import roc_curve, auc

fpr, tpr, _ = roc_curve([0, 0, 1, 1], [0.1, 0.4, 0.35, 0.8])  # Compute ROC
roc_auc = auc(fpr, tpr)                        # Area under curve
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')                # Diagonal line
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.title("ROC Curve")
plt.legend()
plt.show()
```

Saving Plots

```
plt.savefig("figure.png", dpi=300, bbox_inches='tight')  # Save plot to file
```

Show & Clear

```
plt.show()      # Show plot window
plt.clf()       # Clear current figure (useful when plotting in loops)
plt.close()     # Close figure window (useful in scripts or GUI apps)
```

More Useful Functions

`plt.fill_between(x, y1, y2)`	Fill area between curves
`plt.axhline(y=value)`	Draw horizontal line at y
`plt.axvline(x=value)`	Draw vertical line at x
`plt.errorbar(x, y, yerr=...)`	Plot with error bars

`plt.twinx()`	Create secondary y-axis
`plt.subplots_adjust(...)`	Manually adjust subplot spacing
`plt.gca()`	Get current axes
`plt.gcf()`	Get current figure

Top Cheatsheet

Python Cheatsheet
Popular Quick Reference

Vim Cheatsheet
Popular Quick Reference

JavaScript Cheatsheet
Popular Quick Reference

Recent Cheatsheet

SBTI Cheatsheet
Latest Quick Reference

Cheatsheet
Latest Quick Reference

Socket io Cheatsheet
Latest Quick Reference

Developer Tools

TableConvert
Convert tables between formats

Share quick reference and cheat sheet for developers.